



CEBU INSTITUTE OF TECHNOLOGY
U N I V E R S I T Y

IT342-[Section] System Integration and Architecture

System Design Document (SDD)

Project Title: Vault-Tech (Note: New Title, Old Title: MonoVault)

Prepared By: Olimba, Jay-Z Repuesto

Version: [e.g. 1.0]

Date: February 21, 2025

Status: [Draft/Final]

REVISION HISTORY TABLE

Versio n	Date	Author	Changes Made	Status
0.1	21/02/26	Olimba, Jay-Z R.	Added a “dead mans switch”	Draft
0.2	[Date]	[Your Name]	Added API specifications	Review
0.3	[Date]	[Your Name]	Updated database design	Review
0.4	[Date]	[Your Name]	Added UI/UX designs	Review
0.5	[Date]	[Your Name]	Incorporated feedback	Revised
0.6	[Date]	[Your Name]	Final review and corrections	Final
1	[Date]	[Your Name]	Baseline version for development	Approved

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY	4
1.0 INTRODUCTION	5
2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION	5
3.0 NON-FUNCTIONAL REQUIREMENTS	8
4.0 SYSTEM ARCHITECTURE	9
5.0 API CONTRACT & COMMUNICATION	10
Authentication Endpoints	11
User Registration	11
User Login	11
6.0 DATABASE DESIGN	13
7.0 UI/UX DESIGN	14
8.0 PLAN	17

EXECUTIVE SUMMARY

1.1 Project Overview

Vault-Tech is a digital legacy and time-capsule platform that allows users to securely store and schedule the future delivery of digital assets (documents and images). The system enables users to create multiple personalized vaults, assign designated recipients, and trigger the release of these vaults either on a specific future date or via a "Dead Man's Switch" inactivity monitor. The system features a Spring Boot backend API handling secure local storage uploads and automated scheduling, integrated with a React web application and a native Android mobile app.

1.2 Objectives

1. Develop a secure digital time-capsule MVP for storing files and automating their future release.
2. Implement a three-tier architecture utilizing Spring Boot (backend), ReactJS (web frontend), and Android Kotlin (mobile).
3. Design automated background processes (cron jobs) to monitor vault expiration dates and trigger recipient notifications.
4. Implement secure file storage on the server's local file system for the MVP.
5. Establish a subscription-based access model that determines vault lifespans and extension capabilities.

1.3 Scope

Included Features:

- User registration and authentication.
- Creation and management of multiple independent vaults per user.
- File upload capability (PDFs and images) utilizing local server storage.
- Recipient management (assigning names and emails to specific vaults).
- Dual-trigger release mechanisms: Specific Date selection and "Dead Man's Switch" inactivity check-ins.
- Automated email notification system to send secure, time-limited download links to recipients upon vault opening.
- Automated email reminders to vault creators for subscription renewals and check-ins.
- Native Android mobile application and responsive web interface.

- Relational database schema (MySQL) for managing users, metadata, and file paths.

Excluded Features:

- Video file uploading or in-app media streaming.
- Direct cloud object storage integration (deferred to future iterations).
- Real payment gateway integration (subscription logic will be mocked/simulated for the MVP).
- Legally binding document generation (the system acts like a will for digital assets, but does not replace legal counsel).
- Zero-knowledge end-to-end encryption (standard transit and at-rest encryption will be used instead of complex client-side encryption for the MVP).

1.0 INTRODUCTION

1.1 Purpose

This document serves as the comprehensive design specification for the Vault-Tech system. It provides detailed functional requirements, architectural decisions, database design, and the implementation roadmap. This documentation will guide the development process, ensuring that the web, mobile, and backend components, along with the automated scheduling services and secure file storage mechanisms, function cohesively to deliver a secure digital legacy experience.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: Vault-Tech

Domain: Secure File Storage / Digital Legacy

Primary Users:

1. Vault Creators (Users, creates vault)
2. Designated Recipients (Beneficiaries receiving data)
3. Administrators (System managers handling accounts and platform metrics)

Problem Statement: Users lack a secure, reliable, and automated way to pass on digital assets (documents, images, final messages) to specific individuals after a

certain date or in the event of their passing. Furthermore, the platform requires secure oversight to manage users without compromising the strict privacy of the stored digital assets.

Solution: A digital time-capsule platform that allows users to upload files to secure server storage, assign recipients, and set automated release triggers (Specific Date or Dead Man's Switch). The system includes a secure admin panel for system oversight and user management using strict role-based access control

2.2 Core User Journeys

Journey 1: Creating a Time-Date Vault (Customer)

1. User registers/logs in via the web or mobile app.
2. Clicks "Create New Vault" and names it.
3. Selects the "Specific Date" trigger and sets a future unlock date.
4. Adds recipient names and email addresses.
5. Uploads files (which the backend saves securely to the server's local file system).
6. Saves and activates the vault.

Journey 2: Setting up a Dead Man's Switch (Customer)

1. User creates a new vault and selects the "Dead Man's Switch" trigger.
2. Configures recipient details and uploads files.
3. Receives a monthly automated "Check-in" email.
4. Clicks the "I'm Okay" link in the email to reset the inactivity timer.

Journey 3: Automated Vault Unlocking (System & Beneficiary)

1. Backend cron job runs at midnight and identifies vaults scheduled to open.
2. System generates secure, temporary download links for each file stored on the server.
3. System emails the designated beneficiaries a notification with the secure links.
4. Beneficiary clicks the links and downloads the files directly from the secure server endpoint.

Journey 4: System Administration (Administrator)

1. Admin logs into the web platform using administrative credentials.
2. Navigates to the Admin Dashboard to view high-level metrics (e.g., active users, pending vaults, delivery success rates).
3. Searches for a specific user to investigate a reported issue.

4. Suspends an account for violating terms of service or manually adjusts subscription tiers.

2.3 Feature List (MoSCoW)

MUST HAVE

1. User authentication (register, login, JWT security).
2. Multiple vault creation and management per user.
3. Secure local file storage for uploaded documents and images.
4. Recipient management (assigning emails to specific vaults).
5. Automated background scheduler (cron jobs) for trigger evaluation.
6. Automated email delivery of secure download links.
7. Secure Admin Dashboard for user account management (suspend/activate).

SHOULD HAVE

1. Dead Man's Switch inactivity monitor and check-in system.
2. Dashboard tracking active, pending, and delivered vaults for customers.
3. Responsive web interface and native Android application.

COULD HAVE

1. Subscription tier tracking (Free vs. Premium limits).
2. In-app push notifications for Android users.
3. File uploading with direct integration to Cloud Object Storage (e.g., Render) as a future scalability upgrade.

WON'T HAVE

1. Video file uploading.
2. Real payment gateway integration (subscription logic will be simulated for MVP).
3. Legally binding document generation.
4. Admin access to view, download, or edit the actual files inside any user's vault.

2.4 Detailed Feature Specifications

Feature: User Authentication & Profile

- **Screens:** Registration, Login, Profile Dashboard.
- **Data:** Email, Password, Last Active Date, Role (ROLE_USER or ROLE_ADMIN).
- **API Endpoints:** POST /api/auth/register, POST /api/auth/login.

- **Security:** JWT tokens, password hashing (bcrypt).

Feature: Vault Management

- **Screens:** Vault Dashboard, Vault Builder.
- **Functions:** Create vault, select trigger type, add recipients.
- **API Endpoints:** GET /api/vaults, POST /api/vaults, PUT /api/vaults/{id}.

Feature: File Management (Local Storage)

- **Screens:** Drag-and-drop file upload interface (images and PDFs).
- **Functions:** Frontend sends file to Spring Boot -> Spring Boot saves to a secure directory on the server -> Spring Boot saves the local file path to the MySQL database.
- **API Endpoints:** POST /api/vaults/{vaultId}/files, DELETE /api/vaults/{vaultId}/files/{fileId}.

Feature: Admin Control Panel

- **Screens:** System Dashboard, User Management Table, Audit Logs.
- **Functions:** View platform statistics, manage user accounts (suspend/ban), view automated delivery logs.
- **API Endpoints:** GET /api/admin/users, PUT /api/admin/users/{id}/status, GET /api/admin/metrics.
- **Security:** Endpoints require a JWT containing the ROLE_ADMIN authority.

2.5 Acceptance Criteria

AC-1: Successful Vault Creation

- **Given** I am a logged-in user,
- **When** I fill out the vault details, attach a recipient, upload an image or PDF, and click "Activate",
- **Then** the file should be securely stored on the server's file system, the metadata saved in the database, and the vault should appear as "Active" on my dashboard.

AC-2: Dead Man's Switch Reset

- **Given** I have an active vault with an inactivity trigger,
- **When** I receive my monthly check-in email and click the confirmation link,
- **Then** my last_active_date should be updated in the database, preventing the vault from triggering.

AC-3: Automated Delivery Execution

- **Given** a vault's specific unlock date matches the current system date,
- **When** the daily backend scheduler runs,
- **Then** the system should generate secure, temporary download links mapped to the local files, and successfully send them to all assigned recipient email addresses, marking the vault status as "Delivered."

AC-4: Secure Admin Access and User Management

- **Given** I am an Administrator in the User Management dashboard,
- **When** I click "Suspend" on a specific user account,
- **Then** that user's status updates to "Suspended" in the database, and their application access is immediately revoked.
- **And** if a standard user attempts to hit the admin API endpoints, they receive a 403 Forbidden response.

AC-5: Strict Data Privacy

- **Given** I am logged in as an Administrator viewing a user's profile,
- **When** I navigate their account details,
- **Then** I can see their vault count and storage usage,
- **But** the interface and API strictly prevent me from generating download URLs for their files or viewing their vault contents.

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time: ≤ 2 seconds for 95% of requests
- Web page load time: ≤ 3 seconds on broadband
- Mobile app cold start: ≤ 3 seconds
- Support 100 concurrent users
- Database queries complete within 500ms

3.2 Security Requirements

- HTTPS for all communications
- JWT token authentication
- Password hashing with bcrypt (salt rounds = 12)
- SQL injection prevention
- XSS protection
- Rate limiting: 100 requests/minute per IP
- Admin endpoints require role verification

3.3 Compatibility Requirements

- **Web Browsers:** Chrome, Firefox, Safari, Edge (latest 2 versions)
- **Android:** API Level 24+ (Android 7.0+)
- **Screen Sizes:** Mobile (360px+), Tablet (768px+), Desktop (1024px+)
- **Operating Systems:** Windows 10+, macOS 10.15+, Linux Ubuntu 20.04+

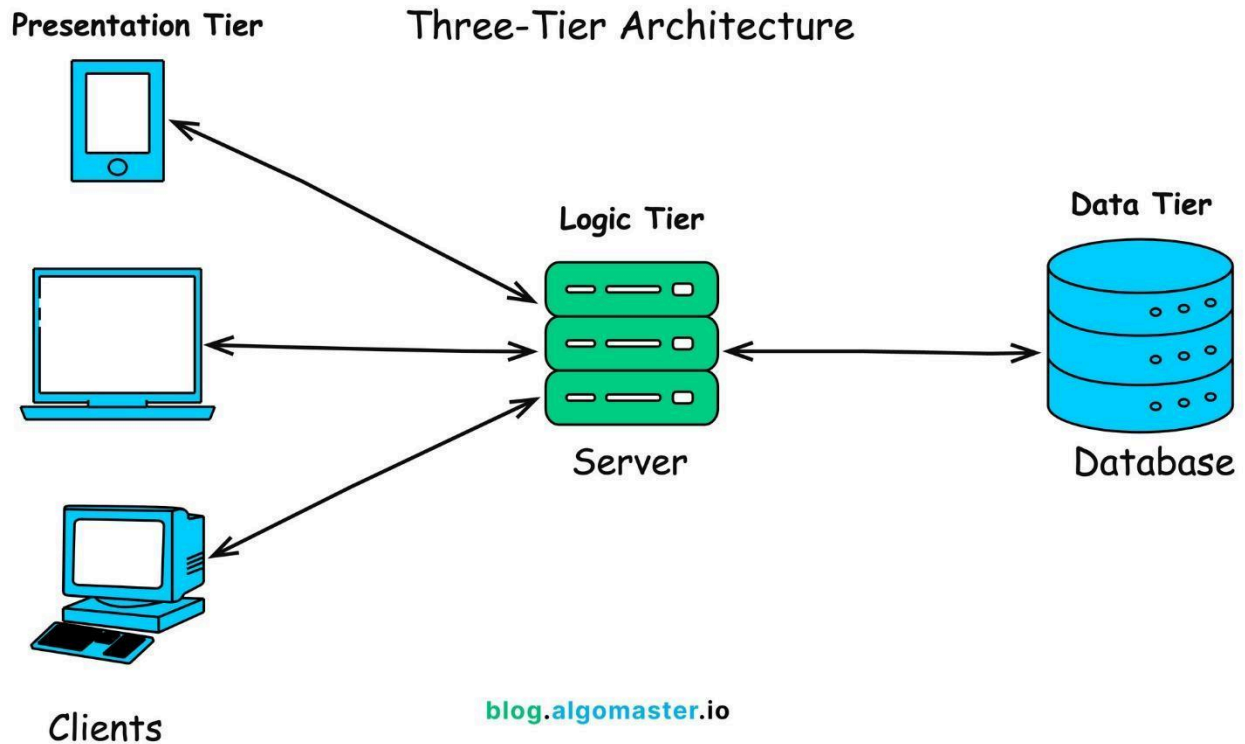
3.4 Usability Requirements

- Complete first purchase within 5 minutes for new users
- WCAG 2.1 Level AA compliance for web
- Consistent navigation across all pages
- Clear error messages with recovery options
- Touch targets minimum 44x44px on mobile
- Keyboard navigation support

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram

Note: This should be a component diagram



Technology Stack:

- **Backend:** Java 17, Spring Boot 3.x, Spring Security, Spring Data JPA
- **Database:** PostgreSQL 14+
- **Web Frontend:** React 18, TypeScript, Tailwind CSS, Axios
- **Mobile:** Kotlin, Jetpack Compose, Retrofit, Room
- **Build Tools:** Maven (Backend), npm/yarn (Web), Gradle (Android)
- **Deployment:** Railway/Heroku (Backend), Vercel/Netlify (Web), APK (Mobile)

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	https://[server_hostname]:[port]/api/v1
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) in Authorization header
Response Structure	<pre>{ "success": boolean, "data": object null, "error": { "code": string, "message": string, } }</pre>

	<pre> "details": object null }, "timestamp": string } </pre>
--	--

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	User Registration
API URL	/auth/register
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	<pre> { "email": <email>, "password": <password>, "firstname": <firstname>, "lastname": <lastname>, } </pre>
Response Structure	<pre> { "success": boolean, "data": { user { "email": <email>, "firstname": <firstname>, "lastname": <lastname>, }, "accessToken": <token>, "refreshToken": <token>, }, "error": { "code": string, "message": string, "details": object null }, "timestamp": string } </pre>

User Login

Description	User Login
API URL	/auth/login
HTTP Method	POST

Format	JSON for all requests/responses
Authentication	None
Request Payload	<pre>{ "email": <email>, "password": <password>, }</pre>
Response Structure	<pre>{ "success": boolean, "data": { "user": { "email": <email>, "firstname": <firstname>, "lastname": <lastname>, "role": <role>, }, "accessToken": <token>, "refreshToken": <token>, }, "error": { "code": string, "message": string, "details": object null }, "timestamp": string }</pre>

5.3 Error Handling

HTTP Status Codes

- 200 OK - Successful request
- 201 Created - Resource created
- 400 Bad Request - Invalid input
- 401 Unauthorized - Authentication required/failed
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist
- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

Error Code Examples

Example 1	<pre>json { "success": false,</pre>
-----------	---------------------------------------

	<pre> "data": null, "error": { "code": "AUTH-001", "message": "Invalid credentials", "details": "Email or password is incorrect" }, "timestamp": "2024-01-28T10:30:00Z" } </pre>
Example 2	<pre> { "success": false, "data": null, "error": { "code": "VALID-001", "message": "Validation failed", "details": { "email": "Email is required", "password": "Must be at least 8 characters" } }, "timestamp": "2024-01-28T10:30:00Z" } </pre>

Common Error Codes

- AUTH-001: Invalid credentials
- AUTH-002: Token expired
- AUTH-003: Insufficient permissions
- VALID-001: Validation failed
- DB-001: Resource not found
- DB-002: Duplicate entry
- BUSINESS-001: Insufficient stock
- SYSTEM-001: Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram

Note: This should be an ERD (Database Schema)

Detailed Relationships:

- **One-to-One:** User ↔ Cart (Each user has exactly one cart)

- **One-to-Many:** User → Orders (User can have multiple orders)
- **One-to-Many:** Cart → CartItems (Cart contains multiple items)
- **One-to-Many:** Order → OrderItems (Order contains multiple items)
- **Many-to-One:** CartItems → Product (Items reference products)
- **Many-to-One:** OrderItems → Product (Items reference products)

Key Tables:

1. **users** - User accounts and authentication
2. **products** - Product catalog information
3. **carts** - Shopping cart per user
4. **cart_items** - Items in shopping cart
5. **orders** - Customer orders
6. **order_items** - Items in each order
7. **refresh_tokens** - JWT refresh tokens

Table Structure Summary:

- **users:** id, email, password_hash, full_name, role, created_at
- **products:** id, name, description, price, stock, image_url, category
- **carts:** id, user_id, created_at
- **cart_items:** id, cart_id, product_id, quantity
- **orders:** id, order_number, user_id, total, status, shipping_address
- **order_items:** id, order_id, product_id, product_name, quantity, price

7.0 UI/UX DESIGN

7.1 Web Application Wireframes

Note: This should be wireframes from Figma

Homepage (Product Listing)

Header: [Logo] [Search Bar] [Cart Icon] [User Menu]

Content: Product Grid (3 columns desktop)

Each Product Card: Image, Name, Price, "Add to Cart" button

Footer: Links, Copyright

Product Detail Page

Back Button

Product Image (large)

Product Name and Price

Description

Quantity Selector (1-10)

"Add to Cart" and "Buy Now" buttons

Product Specifications

Shopping Cart Page

Cart Title

List of Cart Items (Image, Name, Quantity, Price, Remove)

Order Summary: Subtotal, Shipping, Tax, Total

"Continue Shopping" and "Proceed to Checkout" buttons

Checkout Page

Shipping Address Form

Order Review (Items, Prices, Totals)

"Place Order" button

Terms and Conditions note

Admin Dashboard

Sidebar Navigation: Dashboard, Products, Orders, Users

Product Management: Add New button, Product list with Edit/Delete

Order Management: Order list with status filters

7.2 Mobile Application Wireframes

Note: This should be wireframes from Figma

Bottom Navigation

[ Home] [ Search] [ Cart] [ Profile]

Home Screen

Search Bar

Product Grid (2 columns)

Swipe gestures for quick actions

Pull to refresh

Product Detail Screen

Back arrow

Product image (swipeable gallery)

Product info

Quantity selector

"Add to Cart" fixed bottom button

Cart Screen

Edit mode for quantity updates

Swipe to remove items

Order summary sticky bottom

Checkout button

Checkout Flow

Step indicator: Cart → Shipping → Payment → Confirm

Address form (auto-complete)

Order summary

Place order button

Mobile-Specific Features:

- Touch-optimized buttons (min 44x44px)
- Gesture support (swipe, pull-to-refresh)

- Offline caching for product images
- Bottom navigation for main actions
- Simplified forms for mobile input

Design System:

- **Colors:** Primary (#2563EB), Secondary (#7C3AED), Success (#10B981), Error (#EF4444)
- **Typography:** Inter font family, responsive sizing
- **Spacing:** 8px grid system
- **Components:** Consistent buttons, inputs, cards, modals
- **Responsive:** Mobile-first approach, breakpoints at 640px, 768px, 1024px

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1-2: Project setup and documentation

Day 3-4: Complete FRS and NFR

Day 5-7: System architecture design

Week 2: Detailed Design

Day 1-2: API specification

Day 3-4: Database design

Day 5-6: UI/UX wireframes

Day 7: Implementation plan finalization

Phase 2: Backend Development (Week 3-4)

Week 3: Foundation

Day 1: Spring Boot setup with dependencies

Day 2: Database configuration and entities

Day 3: JWT authentication implementation

Day 4: User management endpoints

Day 5: Product CRUD operations

Week 4: Core Features

Day 1: Cart functionality

Day 2: Order management

Day 3: Search and filtering

Day 4: Error handling and validation

Day 5: API documentation and testing

Phase 3: Web Application (Week 5-6)

Week 5: Frontend Foundation

Day 1: React setup with TypeScript

Day 2: Authentication pages (login, register)

Day 3: Product listing page

Day 4: Product detail page

Day 5: Shopping cart implementation

Week 6: Complete Web Features

Day 1: Checkout flow

Day 2: Order history and confirmation

Day 3: Admin dashboard

Day 4: Responsive design polish

Day 5: API integration and testing

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Foundation

- Day 1: Android Studio setup and project structure
- Day 2: Authentication screens
- Day 3: Product browsing
- Day 4: Shopping cart
- Day 5: API service layer

Week 8: Complete Mobile App

- Day 1: Checkout flow
- Day 2: Order management
- Day 3: UI polish and animations
- Day 4: Testing on emulator/device
- Day 5: APK generation and documentation

Phase 5: Integration & Deployment (Week 9-10)

Week 9: Integration Testing

- Day 1: End-to-end testing across platforms
- Day 2: Bug fixes and optimization
- Day 3: Security review
- Day 4: Performance testing
- Day 5: Documentation updates

Week 10: Deployment

- Day 1: Backend deployment (Railway/Heroku)
- Day 2: Web app deployment (Vercel/Netlify)

Day 3: Mobile APK distribution

Day 4: Final testing

Day 5: Project submission

Milestones:

- **M1 (End Week 2):** All design documents complete
- **M2 (End Week 4):** Backend API fully functional
- **M3 (End Week 6):** Web application complete
- **M4 (End Week 8):** Mobile application complete
- **M5 (End Week 10):** Full system deployed and integrated

Critical Path:

1. Authentication system (Week 3)
2. Product catalog API (Week 3-4)
3. Shopping cart functionality (Week 4)
4. Checkout process (Week 6)
5. Cross-platform testing (Week 9)

Risk Mitigation:

- Start with simplest working version of each feature
- Test integration points early and often
- Keep backup of working versions
- Focus on core functionality before enhancements